

MP2.1: A Scalable Tiny SNS Design Document

Phani Jyothi Kurada

UIN: 335006950

CSCE 662

October 29, 2025

1 Introduction

The objective of Machine Problem 2.1 (MP2.1) is to extend the single-server social networking system (SNS) developed in MP1 into a scalable, distributed system. In MP1, the SNS architecture involved a single centralized server handling all client requests for login, follow, unfollow, list, and timeline operations. While functional, that design lacked scalability and fault tolerance.

MP2 introduces a **Coordinator** component that enables dynamic routing between clients and multiple SNS servers. Each SNS server registers itself with the Coordinator through periodic heartbeat messages. The Coordinator maintains a registry of active servers and their cluster information, which allows it to assign clients to appropriate servers during the login phase.

The updated design allows for:

- **Dynamic server registration:** Servers send periodic heartbeats to the Coordinator to indicate their availability.
- **Client redirection:** Clients no longer connect directly to an SNS server. Instead, they query the Coordinator to obtain their assigned server's hostname and port.
- **Fault tolerance:** If a server becomes unavailable, the Coordinator can detect it through missed heartbeats and reroute clients to another active server.
- **Improved scalability:** Multiple SNS servers can handle different subsets of users concurrently, allowing the system to support a larger user base.

Thus, MP2 builds upon MP1's core SNS functionality but evolves it into a **distributed, fault-tolerant, and coordinated system**. The project ensures that the SNS infrastructure can scale horizontally while maintaining consistency across client interactions.

2 Environment Setup

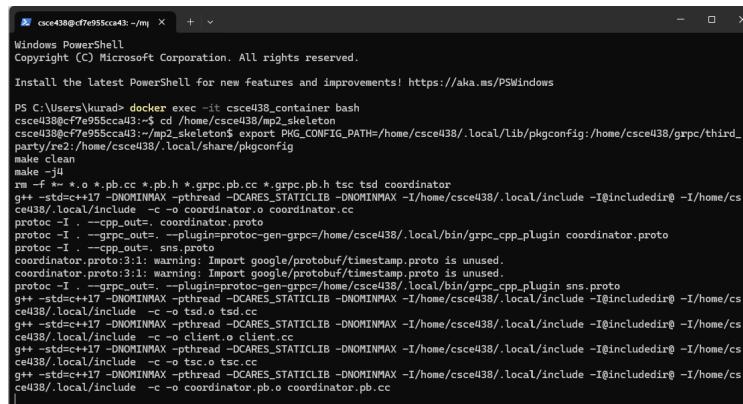
The SNS system was developed and executed inside a Docker container to ensure a consistent environment. A preconfigured image `liuyidockers/csce438_env:latest` was used. The container `csce438_container` was started using the Docker Desktop engine, which manages the container runtime. Once the container was running, the interactive terminal was accessed with the following command:

```
$docker exec -it csce438_container bash
```

This opened a shell inside the running container, from which the server and client programs were compiled and executed. The environment provides the necessary dependencies such as gRPC, protobuf, and glog required for implementing distributed communication between Coordinator, Server, and Client components.

The project was compiled and executed inside the Docker container using the following commands:

```
cd /home/csce438/mp2_skeleton
export PKG_CONFIG_PATH=/home/csce438/.local/lib/pkgconfig:/home/csce438
/grpc/third_party/re2:/home/csce438/.local/share/pkgconfig
make clean
make -j4
```



```
Windows PowerShell
Copyright (C) Microsoft Corporation. All rights reserved.

Install the latest PowerShell for new features and improvements! https://aka.ms/PSWindows

PS C:\Users\kaurad> docker exec -it csce438_container bash
csce438@cf7e955cca43:~$ cd /home/csce438/mp2_skeleton
csce438@cf7e955cca43:~/mp2_skeleton$ export PKG_CONFIG_PATH=/home/csce438/.local/lib/pkgconfig:/home/csce438/grpc/third_
party/re2:/home/csce438/.local/share/pkgconfig
csce438@cf7e955cca43:~/mp2_skeleton$ make clean
csce438@cf7e955cca43:~/mp2_skeleton$ make -j4
rm -f *.o *.pb.cc *.pb.h *.grpc.pb.cc *.grpc.pb.h tsc tsd coordinator
g++ -std=c++17 -DNOMINMAX -pthread -DCARES_STATICLIB -DNOMINMAX -I/home/csce438/.local/include -I@includedir@ -I/home/cs
ce438/.local/include -c -o coordinator.o coordinator.cc
protoc -I . -I /home/csce438/.local/include -I@includedir@ -I/home/csce438/.local/include -I@includedir@ -I/home/cs
ce438/.local/include --cpp_out=. --plugin=protoc-gen-grpc=/home/csce438/.local/bin/grpc_cpp_plugin coordinator.proto
protoc -I . -I /home/csce438/.local/include -I@includedir@ -I/home/csce438/.local/include -I@includedir@ -I/home/cs
ce438/.local/include --cpp_out=. coordinator.proto
coordinator.proto:3:1: warning: Import google/protobuf/timestamp.proto is unused.
coordinator.proto:3:1: warning: Import google/protobuf/timestamp.proto is unused.
g++ -std=c++17 -DNOMINMAX -pthread -DCARES_STATICLIB -DNOMINMAX -I/home/csce438/.local/include -I@includedir@ -I/home/cs
ce438/.local/include -c -o tsd.o tsd.cc
g++ -std=c++17 -DNOMINMAX -pthread -DCARES_STATICLIB -DNOMINMAX -I/home/csce438/.local/include -I@includedir@ -I/home/cs
ce438/.local/include -c -o client.o client.cc
g++ -std=c++17 -DNOMINMAX -pthread -DCARES_STATICLIB -DNOMINMAX -I/home/csce438/.local/include -I@includedir@ -I/home/cs
ce438/.local/include -c -o tsc.o tsc.cc
g++ -std=c++17 -DNOMINMAX -pthread -DCARES_STATICLIB -DNOMINMAX -I/home/csce438/.local/include -I@includedir@ -I/home/cs
ce438/.local/include -c -o coordinator.pb.o coordinator.pb.cc
```

Figure 1: Execution inside Docker Container

The Makefile automates the build process, generating the necessary protobuf and gRPC source files (`.pb.cc` and `.grpc.pb.cc`) and compiling all modules into three executables: `coordinator`, `tsd` (SNS server), and `tsc` (SNS client).

All components communicate over localhost using gRPC channels:

- Coordinator: localhost:9090
- Server: localhost:3010 (or assigned port)

- **Client:** Dynamically connected via Coordinator routing

The Coordinator monitors server health using periodic heartbeat messages, ensuring the client is always connected to an active SNS server.

3 System Components

The system architecture is composed of three primary components: the **Coordinator**, the **Server**, and the **Client**. The **Coordinator** acts as the central control entity that manages server registration, monitors server health through heartbeats, and directs clients to the appropriate active server. Each **Server** (also called the SNS Server or TSD) handles user management, message propagation, and the maintenance of follower relationships within its cluster. The **Client** (TSC) connects to the Coordinator to discover an available server and then interacts directly with that server for social networking operations.

Their interaction occurs through two main modes of operation:

- **Command mode**, which handles user requests such as login, list, follow, and unfollow.
- **Timeline mode**, which manages posting and streaming of messages between users and their followers in real time.

3.1 Coordinator Design

The coordinator, implemented in `coordinator.cc`, manages server clusters, monitors their status via heartbeats, and provides clients with the appropriate server connection details to ensure reliable communication and load balancing.

3.1.1 Cluster Management and Heartbeat Tracking

This component maintains active server clusters, checks server liveness through heartbeat timestamps, and provides utility functions for managing server metadata within each cluster.

```
std::mutex v_mutex;
std::vector<std::vector<zNode*>> clusters = { {}, {}, {} };

std::time_t getTimeNow() {
    return std::chrono::system_clock::to_time_t(std::chrono::
        system_clock::now());
}

bool zNode::isActive() {
    if (!missed_heartbeat) return true;
    return (difftime(getTimeNow(), last_heartbeat) < 10);
}

int findServer(const std::vector<zNode*>& v, int id) {
    for (size_t i = 0; i < v.size(); ++i)
```

```

        if (v[i]->serverID == id) return (int)i;
    return -1;
}

```

3.1.2 Heartbeat Monitoring

The `checkHeartbeat()` function runs continuously on the Coordinator to monitor all registered servers. It checks for missed heartbeats and flags servers as inactive if no update is received within 10 seconds.

```

void checkHeartbeat() {
    while (true) {
        v_mutex.lock();
        for (auto& c : clusters) {
            for (auto& s : c) {
                if (difftime(getTimeNow(), s->last_heartbeat) > 10)
                {
                    std::cout << "[Coordinator] missed heartbeat
                        from server " << s->serverID
                        << " (host=" << s->hostname << ", port
                        =" << s->port << ")" << std::endl;
                    if (!s->missed_heartbeat) {
                        s->missed_heartbeat = true;
                        s->last_heartbeat = getTimeNow();
                    }
                }
            }
        }
        v_mutex.unlock();
        sleep(3);
    }
}

```

3.1.3 Coordinator RPC Implementations

The `CoordServiceImpl` class defines two key RPCs: `Heartbeat()` updates or registers active servers within their respective clusters, while `GetServer()` assigns clients to an available server based on their ID and cluster mapping.

```

class CoordServiceImpl final : public CoordService::Service {
public:
    Status Heartbeat(ServerContext* context, const ServerInfo*
        serverinfo, Confirmation* confirmation) override {
        if (!serverinfo) {
            confirmation->set_status(false);
            return Status(grpc::StatusCode::INVALID_ARGUMENT, "no
                serverinfo");
        }
    }
}

```

```

}

std::string type = serverinfo->type();
int clusterId = 1;
bool parsed = false;
try {
    clusterId = std::stoi(type);
    parsed = true;
} catch (...) {
    std::string digits;
    for (char ch : type) if (isdigit(ch)) digits.push_back(
        ch);
    if (!digits.empty()) {
        try { clusterId = std::stoi(digits); parsed = true;
        } catch (...) { parsed=false; }
    }
}

if (!parsed) clusterId = 1;
if (clusterId < 1) clusterId = 1;
if (clusterId > 3) clusterId = 3;

int sid = serverinfo->serverid();
std::string host = serverinfo->hostname();
std::string port = serverinfo->port();

v_mutex.lock();
auto &cluster = clusters[clusterId - 1];
int idx = findServer(cluster, sid);
if (idx >= 0) {
    zNode* zn = cluster[idx];
    zn->hostname = host;
    zn->port = port;
    zn->type = type;
    zn->last_heartbeat = getTimeNow();
    zn->missed_heartbeat = false;
    std::cout << "[Coordinator] Updated heartbeat for
        serverID=" << sid
        << " in cluster=" << clusterId << " (host=" <<
        host << ", port=" << port << ")\n";
} else {
    zNode* zn = new zNode();
    zn->serverID = sid;
    zn->hostname = host;
    zn->port = port;
    zn->type = type;
    zn->last_heartbeat = getTimeNow();
    zn->missed_heartbeat = false;

```

```

        cluster.push_back(zn);
        std::cout << "[Coordinator] Registered serverID=" << sid
            << " in cluster=" << clusterId << " (host=" <<
                host << ", port=" << port << ")\n";
    }
    v_mutex.unlock();

    confirmation->set_status(true);
    return Status::OK;
}

Status GetServer(ServerContext* context, const ID* id,
    ServerInfo* serverinfo) override {
    if (!id) return Status(grpc::StatusCode::INVALID_ARGUMENT, "
        no id");
    int client_id = id->id();
    if (client_id <= 0) return Status(grpc::StatusCode::
        INVALID_ARGUMENT, "client id > 0 required");

    int clusterId = ((client_id - 1) % 3) + 1;

    v_mutex.lock();
    auto &cluster = clusters[clusterId - 1];
    zNode* chosen = nullptr;
    for (auto& s : cluster) {
        if (s->isActive()) { chosen = s; break; }
    }
    if (!chosen && !cluster.empty()) chosen = cluster.front();
    if (!chosen) { v_mutex.unlock(); return Status(grpc::
        StatusCode::NOT_FOUND, "no server for cluster"); }

    serverinfo->set_serverid(chosen->serverID);
    serverinfo->set_hostname(chosen->hostname);
    serverinfo->set_port(chosen->port);
    serverinfo->set_type(chosen->type);
    v_mutex.unlock();
    return Status::OK;
}
};

```

3.1.4 Coordinator Server Execution

The `RunServer()` function initializes and starts the Coordinator's gRPC server. It launches a separate `checkHeartbeat()` thread to continuously monitor the health of all registered servers and listens for incoming heartbeat and client routing requests.

```
void RunServer(std::string port_no) {
```

```

std::thread hb(checkHeartbeat);
hb.detach();

std::string server_address("127.0.0.1:" + port_no);
CoordServiceImpl service;
ServerBuilder builder;
builder.AddListeningPort(server_address, grpc::
    InsecureServerCredentials());
builder.RegisterService(&service);
std::unique_ptr<Server> server(builder.BuildAndStart());
std::cout << "[Coordinator] Server listening on " <<
    server_address << std::endl;
server->Wait();
}

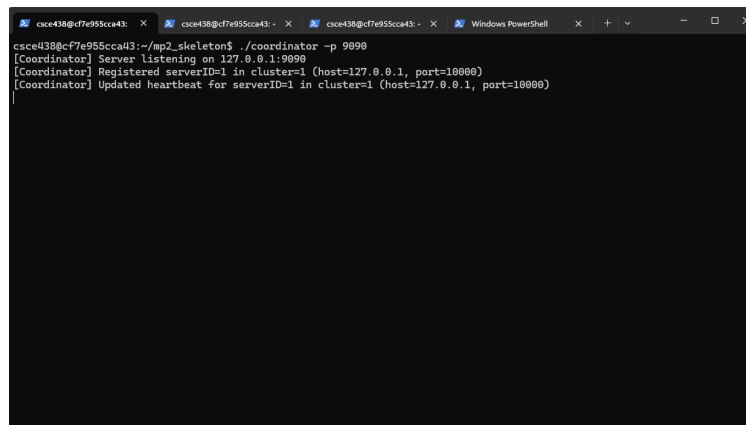
```

The following commands are used to compile and execute the Coordinator inside the Docker container:

```

export PKG_CONFIG_PATH=/home/csce438/.local/lib/pkgconfig:/home/csce438
/grpc/third_party/re2:/home/csce438/.local/share/pkgconfig
make clean
make -j4
./coordinator -p 9090

```



```

csce438@c7e955cca43: X csce438@c7e955cca43: X csce438@c7e955cca43: X Windows PowerShell X + v - □ X
csce438@c7e955cca43:~/mp2_skeleton$ ./coordinator -p 9090
[Coordinator] Server listening on 127.0.0.1:9090
[Coordinator] Registered serverID=1 in cluster=1 (host=127.0.0.1, port=10000)
[Coordinator] Updated heartbeat for serverID=1 in cluster=1 (host=127.0.0.1, port=10000)

```

Figure 2: Coordinator Execution inside Docker Container

3.1.5 Main Function

This function initializes the coordinator by parsing the port argument and starting the server via RunServer(port).

```

int main(int argc, char** argv) {
    std::string port = "9090";
    int opt = 0;
    while ((opt = getopt(argc, argv, "p:")) != -1) {

```

```

        switch (opt) {
            case 'p': port = optarg; break;
            default: std::cerr << "Invalid Command Line Argument\n";
        }
    }
    RunServer(port);
    return 0;
}

```

3.2 Server Side Design

The server is implemented in `tsd.cc` using gRPC for communication with clients. It maintains the state of all connected users, their timelines, and Follow / UnFollow relationships.

3.2.1 Client Data Structure

Each client is represented by the following structure:

```

struct Client {
    std::string username;
    bool connected = true;
    int following_file_size = 0;
    std::vector<Client*> client_followers;
    std::vector<Client*> client_following;
    ServerReaderWriter<Message, Message>* stream = nullptr;
    std::vector<Message> timeline;
    std::unordered_map<std::string, time_t> following_since;

    bool operator==(const Client& c1) const {
        return (username == c1.username);
    }
};

```

The server maintains a global list of all clients:

```
std::vector<Client*> client_db;
```

3.2.2 Client Lookup Function

An essential utility within the server implementation is the `findClient` function. This function enables efficient retrieval of user information by searching the in-memory client database for a specific username. The SNS server relies on this lookup operation in several contexts, including user login, follow and unfollow operations, and timeline message retrieval.

The function performs a linear search over the global `client_db` vector, which stores pointers to all connected clients. For each entry, it compares the stored username with the requested one. If a match is found, the corresponding `Client` object pointer is returned; otherwise, the function returns `nullptr`, indicating that no matching client exists in the current session.

The implementation of this function is shown below:

```
Client* findClient(const std::string& username) {
    for (auto* client : client_db) {
        if (client->username == username)
            return client;
    }
    return nullptr;
}
```

This lookup mechanism serves as a foundational operation for multiple server-side RPCs:

- During the **Login** RPC, it determines whether the user already exists in the database or needs to be newly created.
- In **Follow** and **Unfollow** operations, it retrieves both the requesting user and the target user to update their respective follower and following lists.
- Within the **Timeline** RPC, it identifies the current user and associates their message stream with the active **ServerReaderWriter** instance.

By maintaining a centralized lookup routine, the server ensures consistent access to client metadata, improving code modularity and simplifying the management of user connections within the distributed SNS architecture.

3.2.3 Coordinator Heartbeat and Message Creation

The SNS server maintains a persistent connection with the **Coordinator** to support fault detection and system-level synchronization. To enable this interaction, a set of global variables and helper functions are defined to store and manage coordinator-related state, as well as to facilitate message creation and timestamping.

Coordinator Heartbeat Globals. The following global variables are used to store configuration and runtime metadata necessary for the heartbeat mechanism and server registration:

- **coord_stub**: a gRPC stub used to invoke remote procedure calls on the Coordinator service.
- **coord_host** and **coord_port**: strings storing the hostname and port of the Coordinator, typically set to 127.0.0.1 and 9090 respectively.
- **cluster_id_global** and **server_id_global**: integer identifiers representing the cluster and specific server instance within the distributed SNS network.
- **server_listen_port**: a string indicating the port on which the server listens for incoming client connections.

These parameters enable each server to uniquely identify itself within the system and to periodically send heartbeat signals to the Coordinator. The heartbeat thread invokes the **Heartbeat()** RPC every few seconds, allowing the Coordinator to monitor server liveness and maintain an updated registry of active nodes.

Message Creation Utility. The `MakeMessage()` function provides a standardized way to construct `Message` objects that include user content along with precise timestamps. It accepts a username and message string as inputs, attaches the current system time as a Protocol Buffers timestamp, and returns a fully populated `Message` object ready for transmission. The implementation is shown below:

```
Message MakeMessage(const std::string &username, const std::string &
    msg) {
    Message m;
    m.set_username(username);
    m.set_msg(msg);
    google::protobuf::Timestamp *timestamp = new google::protobuf::
        Timestamp();
    timestamp->set_seconds(time(NULL));
    timestamp->set_nanos(0);
    m.set_allocated_timestamp(timestamp);
    return m;
}
```

This helper function ensures that all posts and timeline messages are timestamped consistently across the distributed system. The timestamps play a critical role in ordering timeline events and ensuring causal consistency during message propagation and retrieval.

3.2.4 RPC Implementations

Login : The Login RPC checks if a user exists in `client_db`. If not, a new client is created; otherwise, it verifies that the user is not already logged in. On success, `connected = true` is set.

```
Status Login(ServerContext* context, const Request* request, Reply*
    reply) override {
    std::string uname = request->username();
    Client* existing = findClient(uname);
    if (!existing) {
        Client* new_client = new Client();
        new_client->username = uname;
        new_client->connected = true;
        client_db.push_back(new_client);
    } else {
        if (existing->connected) {
            reply->set_msg("Login failed: User already logged in");
            return Status(grpc::StatusCode::ALREADY_EXISTS, "User
                already logged in");
        }
        existing->connected = true;
    }
    reply->set_msg("Login successful");
    return Status::OK;
}
```

List : The List RPC retrieves all registered users and followers of the requesting client.

```
Status List(ServerContext* context, const Request* request,
ListReply* list_reply) override {
    Client* current = findClient(request->username());
    if (!current) return Status(grpc::StatusCode::NOT_FOUND, "User
        not found");
    for (auto* c : client_db)
        list_reply->add_all_users(c->username());
    for (auto* f : current->client_followers)
        list_reply->add_followers(f->username());
    return Status::OK;
}
```

Follow : The Follow RPC lets one user follow another and updates both clients' relationship vectors with timestamps.

```
Status Follow(ServerContext* context, const Request* request, Reply*
reply) override {
    std::string follower_name = request->username();
    std::string target_name = request->arguments(0);
    Client* follower = findClient(follower_name);
    Client* target = findClient(target_name);
    if (!follower || !target) {
        reply->set_msg("User not found");
        return Status::OK;
    }
    if (follower == target) {
        reply->set_msg("Cannot follow yourself");
        return Status::OK;
    }
    for (auto* f : follower->client_following) {
        if (f == target) {
            reply->set_msg("Already following");
            return Status::OK;
        }
    }
    follower->client_following.push_back(target);
    target->client_followers.push_back(follower);
    follower->following_since[target->username] = time(NULL);
    reply->set_msg("Followed successfully");
    return Status::OK;
}
```

UnFollow : The UnFollow RPC removes a following relationship and updates related structures.

```
Status UnFollow(ServerContext* context, const Request* request,
Reply* reply) override {
```

```

std::string follower_name = request->username();
std::string target_name = request->arguments(0);
Client* follower = findClient(follower_name);
Client* target = findClient(target_name);
if (!follower || !target) {
    reply->set_msg("User not found");
    return Status::OK;
}
if (follower == target) {
    reply->set_msg("Cannot unfollow yourself");
    return Status::OK;
}
auto it = std::find(follower->client_following.begin(), follower
->client_following.end(), target);
if (it == follower->client_following.end()) {
    reply->set_msg("Not following this user");
    return Status::OK;
}
follower->client_following.erase(it);
follower->following_since.erase(target->username());
auto it2 = std::remove_if(target->client_followers.begin(),
    target->client_followers.end(),
        [&](Client* c) { return c == follower;
        });
target->client_followers.erase(it2, target->client_followers.end
());
reply->set_msg("Unfollowed successfully");
return Status::OK;
}

```

Timeline : The Timeline RPC opens a bidirectional stream, retrieves the 20 most recent posts from followed users, and broadcasts new posts to connected followers.

```

Status Timeline(ServerContext* context, ServerReaderWriter<Message,
Message>* stream) override {
    Client* current_user = nullptr;
    Message init_msg;
    if (!stream->Read(&init_msg))
        return Status(grpc::StatusCode::INVALID_ARGUMENT, "No
        username received");
    current_user = findClient(init_msg.username());
    if (!current_user)
        return Status(grpc::StatusCode::NOT_FOUND, "User not found")
        ;
    current_user->stream = stream;

    std::vector<Message> recent_posts;
    for (auto* followed : current_user->client_following) {

```

```

        auto follow_time = current_user->following_since[followed->
            username];
        for (auto& m : followed->timeline)
            if (m.timestamp().seconds() >= follow_time)
                recent_posts.push_back(m);
    }

    std::sort(recent_posts.begin(), recent_posts.end(),
        [](const Message& a, const Message& b) {
            return a.timestamp().seconds() > b.timestamp().
                seconds();
        });
    if (recent_posts.size() > 20)
        recent_posts.resize(20);
    for (auto& m : recent_posts)
        stream->Write(m);

    Message client_msg;
    while (stream->Read(&client_msg)) {
        current_user->timeline.push_back(client_msg);
        for (auto* follower : current_user->client_followers)
            if (follower->stream)
                follower->stream->Write(client_msg);
    }
    current_user->stream = nullptr;
    return Status::OK;
}

```

3.2.5 Heartbeat Loop

The `heartbeat_loop()` function operates as a background thread that periodically sends heartbeat messages from each SNS server to the Coordinator every five seconds. This ensures continuous monitoring of server health and helps the Coordinator maintain an updated list of active servers.

```

void heartbeat_loop(const std::string &coord_addr) {
    if (!coord_stub) {
        coord_stub = CoordService::NewStub(
            grpc::CreateChannel(coord_addr, grpc::
                InsecureChannelCredentials()));
    }
    while (true) {
        ServerInfo info;
        info.set_serverid(server_id_global);
        info.set_hostname("127.0.0.1");
        info.set_port(server_listen_port);
        info.set_type(std::to_string(cluster_id_global));
    }
}

```

```

        Confirmation conf;
        grpc::ClientContext ctx;
        grpc::Status st = coord_stub->Heartbeat(&ctx, info, &conf);
        if (!st.ok()) {
            std::cerr << "[tsd] Heartbeat RPC failed: "
                      << st.error_message() << std::endl;
        } else {
            // Optional: std::cout << "[tsd] Heartbeat OK\n";
        }
        sleep(5);
    }
}

```

3.2.6 Main Function

The `main()` function serves as the entry point for the server. It parses command-line arguments such as port number, cluster ID, server ID, and coordinator details. After initialization, it sets up logging, starts a heartbeat thread to register the server with the Coordinator, and finally calls `RunServer(port)` to begin serving client requests.

```

int main(int argc, char** argv) {
    std::string port = "3010";
    int opt = 0;
    std::string coord_h = "127.0.0.1";
    std::string coord_k = "9090";
    int cluster = 1;
    int serverid = 1;
    while ((opt = getopt(argc, argv, "p:c:s:h:k:")) != -1) {
        switch (opt) {
            case 'p': port = optarg; break;
            case 'c': cluster = atoi(optarg); break;
            case 's': serverid = atoi(optarg); break;
            case 'h': coord_h = optarg; break;
            case 'k': coord_k = optarg; break;
            default: std::cerr << "Invalid Command Line Argument\n";
        }
    }

    server_listen_port = port;
    cluster_id_global = cluster;
    server_id_global = serverid;
    coord_host = coord_h;
    coord_port = coord_k;
    std::string coord_addr = coord_host + ":" + coord_port;

    std::string log_file_name = std::string("server-") + port;
}

```

```

    google::InitGoogleLogging(log_file_name.c_str());
    log(INFO, "Logging Initialized. Server starting...");
    std::thread hb(heartbeat_loop, coord_addr);
    hb.detach();

    RunServer(port);

    return 0;
}

```

3.2.7 Server Execution

The `main` function serves as the entry point of the server program. It parses command-line arguments for parameters such as port number, cluster ID, server ID, and coordinator details. After initialization, it sets up Google logging, launches a background thread for the heartbeat mechanism, and starts the gRPC server by invoking `RunServer(port)`.

The server listens on a configurable port (default: 3010) and registers the `SNSServiceImpl` service, which handles all SNS-related RPCs from clients.

```

void RunServer(std::string port_no) {
    std::string server_address = "0.0.0.0:" + port_no;
    SNSServiceImpl service;

    ServerBuilder builder;
    builder.AddListeningPort(server_address, grpc::
        InsecureServerCredentials());
    builder.RegisterService(&service);
    std::unique_ptr<Server> server(builder.BuildAndStart());
    std::cout << "Server listening on " << server_address << std::
        endl;
    log(INFO, "Server listening on " + server_address);

    server->Wait();
}

```

The following commands are used to compile and run the server inside the Docker container:

```

docker exec -it csce438_container bash
cd /home/csce438/mp2_skeleton
export PKG_CONFIG_PATH=/home/csce438/.local/lib/pkgconfig:\
/home/csce438/grpc/third_party/re2:/home/csce438/.local/share/pkgconfig
make clean
make -j4

./tsd -c 1 -s 1 -h localhost -k 9090 -p 10000

```

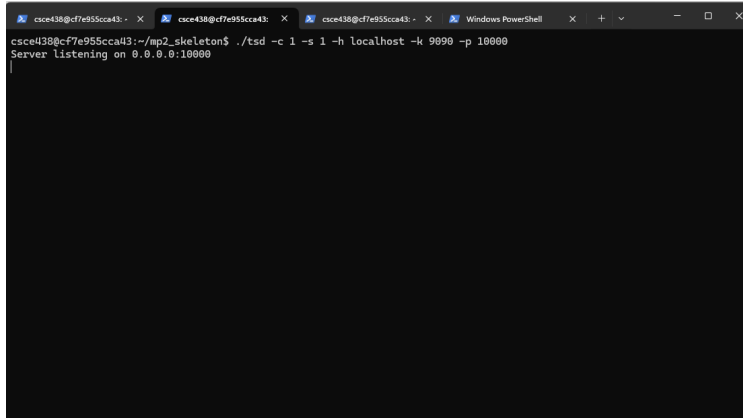


Figure 3: Server Execution inside Docker Container

The figure above shows the server running inside the Docker container, successfully listening on the specified port and ready to handle incoming client requests.

3.3 Client Side Design

The client component of the SNS system is responsible for connecting to the server, sending commands, receiving timeline messages asynchronously, and displaying them to the user. This is implemented in file `tsc.cc`. The key features are:

- Connects to the server via gRPC using the hostname and port provided.
- Sends commands to the server: `LOGIN`, `LIST`, `FOLLOW`, `UNFOLLOW`, `TIMELINE`.
- Receives timeline messages asynchronously from followed users.
- Displays the last 20 messages in reverse chronological order (newest first) from followed users, respecting the `following_since` timestamp.

3.3.1 Message Creation

The function `MakeMessage` creates a new `Message` object with the specified username and message content. It also attaches a timestamp representing the current time using Google Protobuf's `Timestamp`.

```
Message MakeMessage(const std::string &username, const std::string &
msg) {
    Message m;
    m.set_username(username);
    m.set_msg(msg);
    google::protobuf::Timestamp *timestamp = new google::protobuf::
        Timestamp();
    timestamp->set_seconds(time(NULL));
    timestamp->set_nanos(0);
    m.set_allocated_timestamp(timestamp);
```

```
    return m;  
}
```

3.3.2 Client Class Design

The `Client` class defines the client-side logic responsible for user interaction, command parsing, and communication with the SNS server through gRPC calls. It manages connection parameters, invokes RPCs via stubs, and handles timeline streaming asynchronously.

- Maintains connection parameters such as hostname, username, and coordinator port.
- Processes user commands including LOGIN, LIST, FOLLOW, UNFOLLOW, and TIMELINE.
- Communicates with the Coordinator to determine server routing.
- Interfaces with the gRPC stub to send and receive data.

```
class Client : public IClient {  
public:  
    Client(const std::string &coordHost, const std::string &userId,  
           const std::string &coordPort)  
        : coord_host(coordHost), username(userId), coord_port(  
            coordPort) {}  
  
protected:  
    virtual int connectTo();  
    virtual IReply processCommand(std::string &input);  
    virtual void processTimeline();  
  
private:  
    std::string coord_host;  
    std::string username;  
    std::string coord_port;  
    std::unique_ptr<SNSService::Stub> stub_;  
  
    IReply Login();  
    IReply List();  
    IReply Follow(const std::string &username);  
    IReply UnFollow(const std::string &username);  
    IReply Timeline(const std::string &username);  
};
```

Each method corresponds to an RPC call that encapsulates communication with either the Coordinator or the SNS server, returning a structured `IReply` object representing the outcome.

3.3.3 Connecting to the Server

The `connectTo()` function first contacts the Coordinator using the provided host and port to retrieve the assigned SNS server information. It then connects to the designated server and performs a login RPC to establish the session.

```
int Client::connectTo() {
    std::string coord_addr = coord_host + ":" + coord_port;
    auto coord_stub = CoordService::NewStub(grpc::CreateChannel(
        coord_addr, grpc::InsecureChannelCredentials()));

    ID id;
    try {
        id.set_id(std::stoi(username));
    } catch (...) {
        std::cerr << "Client id must be numeric (use -u <id>)\n";
        return -1;
    }

    ServerInfo serverinfo;
    ClientContext ctx;
    Status st = coord_stub->GetServer(&ctx, id, &serverinfo);
    if (!st.ok()) {
        std::cerr << "GetServer failed: " << st.error_message() <<
            std::endl;
        return -1;
    }

    std::string server_host = serverinfo.hostname();
    std::string server_port = serverinfo.port();
    std::string addr = server_host + ":" + server_port;

    stub_ = SNSService::NewStub(grpc::CreateChannel(addr, grpc::
        InsecureChannelCredentials()));

    IReply login_reply = Login();
    if (!login_reply.grpc_status.ok() || login_reply.comm_status !=
        SUCCESS) {
        std::cerr << "Login failed: " << login_reply.grpc_status.
            error_message() << std::endl;
        return -1;
    }
    std::cout << "Login successful as " << username << " connected
        to " << addr << std::endl;
    return 1;
}
```

The following commands are used to compile and launch a client that connects to the Coordinator and logs in to its assigned server:

```

docker exec -it csce438_container bash
cd /home/csce438/mp2_skeleton
export PKG_CONFIG_PATH=/home/csce438/.local/lib/pkgconfig:\
/home/csce438/grpc/third_party/re2:/home/csce438/.local/share/pkgconfig
make clean
make -j4
./tsc -h localhost -k 9090 -u 1

```

```

csce438@c7e955cca43: /mp2_skeleton$ ./tsc -h localhost -k 9090 -u 1
Logging Initialized. Client starting...
Login successful as 1 connected to 127.0.0.1:9090

===== TINY SNS CLIENT =====
Command Lists and Format:
FOLLOW <username>
UNFOLLOW <username>
LIST
TIMELINE
=====

Cmd> LIST
Command completed successfully
All users: 1,
Followers:
Cmd> TIMELINE
Command completed successfully
Now you are in the timeline
p1
p2
p3

```

Figure 4: Client Creation and Login Execution

3.3.4 Command Processing

The `processCommand()` function parses the user's input command and executes the corresponding RPC operation. If the user enters the `TIMELINE` command, it checks the server connection and switches to timeline mode only if the server is active; otherwise, it returns an invalid command response.

```

IReply Client::processCommand(std::string &input) {
    IReply ire;
    std::string cmd, arg;
    size_t space = input.find(" ");
    if (space != std::string::npos) {
        cmd = input.substr(0, space);
        arg = input.substr(space + 1);
    } else {
        cmd = input;
    }

    if (cmd == "FOLLOW") {
        ire = Follow(arg);
    } else if (cmd == "UNFOLLOW") {
        ire = UnFollow(arg);
    } else if (cmd == "LIST") {
        ire = List();
    }
}

```

```

    } else if (cmd == "TIMELINE") {

        grpc::ClientContext ctx;
        auto health_stub = SNSService::NewStub(
            grpc::CreateChannel(coord_host + ":" + coord_port,
                               grpc::InsecureChannelCredentials()));

        Request req;
        req.set_username(username);
        ListReply dummy;
        Status st = stub_->List(&ctx, req, &dummy);

        if (st.ok()) {
            ire.comm_status = SUCCESS;
            ire.grpc_status = grpc::Status::OK;
        } else {
            ire.comm_status = FAILURE_INVALID;
            ire.grpc_status = grpc::Status::OK;
        }
    }

else {
    ire.comm_status = FAILURE_INVALID;
    ire.grpc_status = grpc::Status::OK;
}

    return ire;
}

```

3.3.5 Client RPC Methods

The client class defines methods corresponding to each server RPC. Each function uses gRPC to communicate with the server and returns an IReply object containing both gRPC and command-level status.

List RPC : The List method sends the client's username to the server and retrieves all registered users and followers, storing the result in vectors.

```

IReply Client::List() {
    IReply ire;
    Request req;
    req.set_username(username);
    ClientContext context;
    ListReply list_reply;

    Status status = stub_->List(&context, req, &list_reply);

    if (!status.ok()) {

```

```

        ire.grpc_status = grpc::Status::OK;
        ire.comm_status = FAILURE_INVALID;
        return ire;
    }

    ire.grpc_status = grpc::Status::OK;
    ire.comm_status = SUCCESS;

    for (int i = 0; i < list_reply.all_users_size(); i++)
        ire.all_users.push_back(list_reply.all_users(i));

    for (int i = 0; i < list_reply.followers_size(); i++)
        ire.followers.push_back(list_reply.followers(i));

    return ire;
}

```

Follow RPC : The Follow method sends a follow request and updates the client's status based on the server's reply message.

```

IReply Client::Follow(const std::string &username2) {
    IReply ire;
    ClientContext context;
    Request req;
    Reply rep;

    req.set_username(username);
    req.add_arguments(username2);

    Status status = stub_->Follow(&context, req, &rep);
    ire.grpc_status = status;

    std::string msg = rep.msg();

    if (msg == "Followed successfully") {
        ire.comm_status = SUCCESS;
    } else if (msg == "Already following") {
        ire.comm_status = FAILURE_ALREADY_EXISTS;
    } else if (msg == "Cannot follow yourself") {
        ire.comm_status = FAILURE_ALREADY_EXISTS;
    } else if (msg == "User not found") {
        ire.comm_status = FAILURE_INVALID_USERNAME;
    } else {
        ire.comm_status = FAILURE_UNKNOWN;
    }

    return ire;
}

```

```
}
```

UnFollow RPC : The UnFollow method sends a request to stop following a user and sets the result based on the server's response.

```
IReply Client::UnFollow(const std::string &username2) {
    IReply ire;
    ClientContext context;
    Request req;
    Reply rep;

    req.set_username(username);
    req.add_arguments(username2);

    Status status = stub_->UnFollow(&context, req, &rep);
    ire.grpc_status = status;

    std::string msg = rep.msg();

    if (msg == "Unfollowed successfully") {
        ire.comm_status = SUCCESS;
    } else if (msg == "Not following this user") {
        ire.comm_status = FAILURE_NOT_A_FOLLOWER;
    } else if (msg == "Cannot unfollow yourself") {
        ire.comm_status = FAILURE_INVALID_USERNAME;
    } else if (msg == "User not found") {
        ire.comm_status = FAILURE_INVALID_USERNAME;
    } else {
        ire.comm_status = FAILURE_UNKNOWN;
    }

    return ire;
}
```

Login RPC : The Login method authenticates the user by sending a username to the server and sets the reply status based on the login outcome.

```
IReply Client::Login() {
    IReply ire;
    Request req;
    req.set_username(username);

    ClientContext context;
    Reply reply;
    Status status = stub_->Login(&context, req, &reply);
    ire.grpc_status = status;

    if (status.ok() && reply.msg() == "Login successful") {
        ire.comm_status = SUCCESS;
    }
}
```

```

    } else if (reply.msg() == "Username already exists") {
        ire.comm_status = FAILURE_ALREADY_EXISTS;
        std::cerr << reply.msg() << std::endl;
    } else {
        ire.comm_status = FAILURE_UNKNOWN;
    }
    return ire;
}

```

Timeline Streaming : The Timeline method opens a bidirectional gRPC stream, enabling real-time message exchange between the client and followed users through concurrent read/write threads.

```

IReply Client::Timeline(const std::string &username) {
    IReply ire;
    ClientContext context;

    std::shared_ptr<ClientReaderWriter<Message, Message>> stream;
    try {
        stream = stub_->Timeline(&context);
    } catch (...) {
        ire.comm_status = FAILURE_INVALID;
        ire.grpc_status = grpc::Status::OK;
        return ire;
    }

    if (!stream) {
        ire.comm_status = FAILURE_INVALID;
        ire.grpc_status = grpc::Status::OK;
        return ire;
    }

    Message init_msg;
    init_msg.set_username(username);
    if (!stream->Write(init_msg)) {
        ire.comm_status = FAILURE_INVALID;
        ire.grpc_status = grpc::Status::OK;
        return ire;
    }

    ire.comm_status = SUCCESS;
    ire.grpc_status = grpc::Status::OK;

    std::thread reader([stream]() {
        Message server_msg;
        while (stream->Read(&server_msg)) {
            std::time_t post_time = server_msg.timestamp().seconds()

```

```

        ;
        displayPostMessage(server_msg.username(), server_msg.msg
        (), post_time);
    }
});

while (true) {
    std::string text = getPostMessage();
    if (text.empty()) continue;
    Message m = MakeMessage(username, text);
    if (!stream->Write(m)) break;
}

stream->WritesDone();
reader.join();

return ire;
}

void Client::processTimeline() {
    IReply reply = Timeline(username);
    if (reply.comm_status != SUCCESS) {
        std::cout << "Command failed with invalid command" << std::
        endl;
    }
}
}

```

3.3.6 Main Function

The `main()` function initializes parameters, parses command-line arguments, creates a client instance, and starts execution by calling `run()`.

```

int main(int argc, char **argv) {
    std::string coord_host = "localhost";
    std::string coord_port = "9090";
    std::string user_id = "1"; // numeric id required by MP2.1

    int opt = 0;
    while ((opt = getopt(argc, argv, "h:k:u:")) != -1) {
        switch (opt) {
            case 'h': coord_host = optarg; break; // coordinator
            host
            case 'k': coord_port = optarg; break; // coordinator
            port
            case 'u': user_id = optarg; break; // numeric user
            id
            default: std::cout << "Invalid Command Line Argument\n";
        }
    }
}

```

```
}

std::cout << "Logging Initialized. Client starting...\n";
Client myc(coord_host, user_id, coord_port);
myc.run();

return 0;
}
```

3.4 Client-Server Interaction

1. **Login:** The client sends a login request, and the server validates the username before marking the client as connected.
2. **Command Execution:** The client issues commands such as `LIST`, `FOLLOW`, or `UNFOLLOW`, which the server processes and responds to accordingly.
3. **Timeline Streaming:**
 - Displays the latest 20 posts from followed users, ordered from newest to oldest.
 - Continuously updates in real time as new posts are made.

4 Test Case Scenarios

4.1 Test Case 1: Server Registration and Timeline Activation

Objective: Validate successful server registration with the coordinator and verify that the client enters timeline mode correctly after connection.

Procedure: Run the coordinator, start the server to send heartbeat signals, and connect the client to execute `LIST` and `TIMELINE` commands.

Expected Result: Server heartbeat is registered, and the client successfully accesses the timeline mode.

```
csce438@c7e955cca43: X csce438@c7e955cca43: X csce438@c7e955cca43: X Windows PowerShell X + - _
csce438@c7e955cca43:~/mp2_skeleton$ ./coordinator -p 9090
[Coordinator] Server listening on 127.0.0.1:9090
[Coordinator] Registered serverID=1 in cluster=1 (host=127.0.0.1, port=10000)
[Coordinator] Updated heartbeat for serverID=1 in cluster=1 (host=127.0.0.1, port=10000)

csce438@c7e955cca43:~/mp2_skeleton$ ./tsd -c 1 -s 1 -h localhost -k 9090 -p 10000
Server listening on 0.0.0.0:10000

csce438@c7e955cca43:~/mp2_skeleton$ ./tsc -h localhost -k 9090 -u 1
Logging initialized. Client starting...
Login successful as 1 connected to 127.0.0.1:10000

===== TINY SNS CLIENT =====
Command Lists and Format:
FOLLOW <username>
UNFOLLOW <username>
LIST
TIMELINE

Cmd> LIST
Command completed successfully
All users: 1,
Followers:
Cmd> TIMELINE
Command completed successfully
Now you are in the timeline
p1
p2
p3
```

Figure 5: Coordinator, server registration, and client entering timeline mode

4.2 Test Case 2: Server Failure and Reconnection Validation

Objective: Validate that the coordinator detects a failed server and properly handles client reconnection once the server restarts.

Procedure: Start the coordinator and server, kill the server process, attempt client connection during downtime, then restart the server and reconnect the client.

Expected Result: The client fails to connect when the server is down and successfully reconnects once the server resumes, with timeline mode restored.

[illegible]

```

csc438@cf7f955ccad3:~$ csc438x X csc438x X csc438x X csc438x X csc438x X csc438x X Window X + X
csc438@cf7f955ccad3:~$ mp2_skeleton$ ./tsc -h localhost -k 9999 -u 1
Logging Initialized. Client starting...
Login successful as 1 connected to 127.0.0.1:10000

===== TINY SNS CLIENT =====
Command Lists and Format:
FOLLOW <username>
UNFOLLOW <username>
LIST
TIMELINE

Cnd> LIST
Command completed successfully
All users: 1,
Followers:
Cnd> TIMELINE
Command completed successfully
Now you are in the timeline
p1
p2
p
}

csc438@cf7f955ccad3:~$ csc438x X csc438x X csc438x X csc438x X csc438x X csc438x X Window X + X
csc438@cf7f955ccad3:~$ mp2_skeleton$ ./tsc -h localhost -k 9999 -u 2
Logging Initialized. Client starting...
Login successful as 2 connected to 127.0.0.1:10001

===== TINY SNS CLIENT =====
Command Lists and Format:
FOLLOW <username>
UNFOLLOW <username>
LIST
TIMELINE

Cnd> LIST
Command failed
Cnd> TIMELINE
Command failed
Cnd> LIST
Command failed
Cnd> TIMELINE
Command failed
Cnd> Killed
csc438@cf7f955ccad3:~$ mp2_skeleton$ ./tsc -h localhost -k 9999 -u 2
Logging Initialized. Client starting...
Login successful as 2 connected to 127.0.0.1:10001

===== TINY SNS CLIENT =====
Command Lists and Format:
FOLLOW <username>
UNFOLLOW <username>
LIST
TIMELINE

Cnd> LIST
Command completed successfully
All users: 2,
Followers:
Cnd> TIMELINE
Command completed successfully
Now you are in the timeline
p1
p5
p6

```

28

Note: All the commands mentioned above were executed according to the testcase design document.

5 Makefile Modifications from MP1 to MP2

The MP2 Makefile extends the MP1 build system to support the new **Coordinator** component and inter-server communication. Key updates include:

- Added compilation targets for **coordinator**, integrating `coordinator.pb.o` and `coordinator.grpc.o`.
- Updated `tsd` and `tsc` build rules to include coordinator-related protobuf and gRPC files.
- Improved proto generation rules to handle both `.pb.cc/.h` and `.grpc.pb.cc/.h` files.
- Introduced explicit object build rule pattern (`%.o: %.cc`) for cleaner dependency management.
- Ensured consistent environment configuration via `PKG_CONFIG_PATH` for gRPC and protobuf libraries.

Overall, the MP2 Makefile provides a more modular and extensible structure that supports multiple networked services (Coordinator, Server, Client) compared to MP1's single-client-server setup.